

Finite Volume Analysis of Unsteady Couette Flow

Course: Computational Fluid Dynamics for Engineering Applications

Vadhri Venkata Ratam

Date: 20-Feb-2025

[Link to Colab notebook used for results.](#)

1 Introduction

Two very large parallel plates, with an incompressible fluid between them, are separated by a distance h . Initially, both the plates and fluid are at rest. At time $t = 0$, the lower plate located at $y = 0$ is suddenly accelerated and moves with constant velocity U_0 , while the upper plate at $y = h$ remains stationary. The governing equation describing the development of the fluid velocity profile is,

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2} \quad \begin{cases} u(y, t \leq 0) = 0 & \text{for all } y \\ u(0, t) = U_0 & \text{for } t > 0 \\ u(h, t) = 0 & \text{for } t > 0 \end{cases}$$

where u is the fluid velocity parallel to the plates, ν is the kinematic viscosity, t is time, and y is the distance perpendicular to the plates. The analytical solution satisfying these conditions is,

$$u(y, t) = U_0 \left(1 - \frac{y}{h}\right) - \frac{2U_0}{\pi} \sum_{n=1}^{\infty} \frac{1}{n} \sin\left(\frac{n\pi y}{h}\right) \exp\left(-\frac{n^2\pi^2\nu t}{h^2}\right) \quad (1)$$

The figure (1) illustrates the physical model and spatial discretization of unsteady Couette flow between two large parallel plates separated by a distance h . The coordinate system is defined such that the x -direction represents the flow direction, and the y -direction represents the transverse direction across the plate separation. Since the plates are assumed infinitely large, velocity variation occurs only in the y -direction, and the flow is one-dimensional, i.e., $u = u(y, t)$. The dotted horizontal lines represent the velocity values $u_i(t)$ computed at the control volume centers (nodes). These discrete values are used to approximate the continuous velocity profile between the moving lower plate and stationary upper plate.

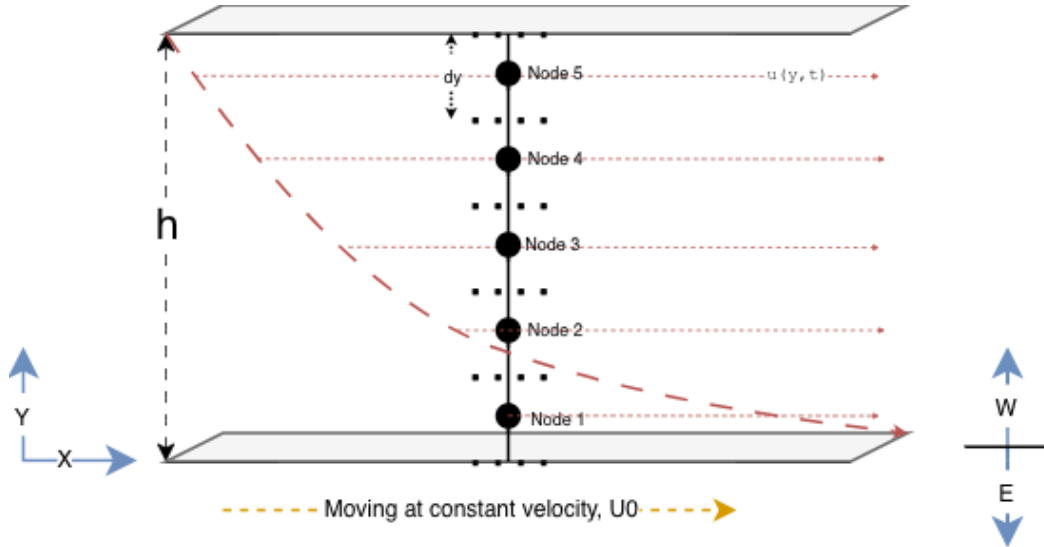


Figure 1: Flow and discretizations abstract

The fluid domain between the plates is discretized into a finite number of control volumes along the y -direction. The nodes shown by solid circles represent the center of each control volume where the velocity is computed. The dotted points indicate the control volume faces. In the finite volume formulation, each node has a west (W) face and an east (E) face corresponding to the lower and upper neighboring control volume interfaces,

respectively.

1.1 Numerical methods

To obtain a numerical solution, the finite volume method is used to integrate the equation over discrete control volumes in space and over finite time intervals. This converts the differential equation into algebraic equations that can be solved step by step to determine the velocity distribution as it evolves with time. The governing equation integrated over a control volume (CV) and over a time interval from t to $t + \Delta t$ is

$$\int_t^{t+\Delta t} \int_{CV} \frac{\partial u}{\partial t} dV dt = \int_t^{t+\Delta t} \int_{CV} \nu \frac{\partial^2 u}{\partial y^2} dV dt$$

The variation of $u(y,t)$ with time can have a varying influence of previous, present and future values w.r.t time. This approach can be generalized by parameterizing the influence of each of them using a parameter θ . The method is explained in detail in Appendix E

1.1.1 Solving the governing equation

Nodes 2,3,4 The steps for solving the governing equation for non-boundary nodes is detailed in the Appendix B. The notable equation that is parameterized with θ is as below.

$$u_P^{n+1} - u_P^n = A [\theta (u_E^{n+1} - 2u_P^{n+1} + u_W^{n+1}) + (1 - \theta) (u_E^n - 2u_P^n + u_W^n)]$$

where:

$$A = \frac{\nu \Delta t}{(\Delta y)^2}$$

Boundary of moving plate The steps for deriving the equation of moving plate at the bottom is at velocity of U_0 is provided in Appendix C. The notable equation parameterized by θ is below with the same value of A above.

$$u_P^{n+1} - u_P^n = 2A [\theta (2U_0 - 3u_P^{n+1} + u_W^{n+1}) + (1 - \theta) (2U_0 - 3u_P^n + u_W^n)]$$

Boundary with static plate The steps for deriving the equation for static plate at the top which is not having any velocity is provided in the Appendix D. The notable equation parametrized by θ is as below with the same value of A above.

$$u_P^{n+1} - u_P^n = 2A [\theta (u_E^{n+1} - 3u_P^{n+1}) + (1 - \theta) (u_E^n - 3u_P^n)]$$

2 Explicit Discretization

2.1 Physical Parameters

Plate spacing $h = 10 \text{ mm} = 0.01 \text{ m}$

Plate velocity: $U_0 = 1 \text{ m/s}$

Fluid properties: $\mu = 10^{-3} \text{ kg/(m.s)}, \rho = 1000 \text{ kg/m}^3$

Kinematic viscosity: $\nu = \frac{\mu}{\rho} = 10^{-6} \text{ m}^2/\text{s}$

2.2 Grid Definition

Number of nodes: $N = 5$

Grid spacing: $\Delta y = \frac{h}{N} = \frac{0.01}{5} = 0.002 \text{ m}$

Node locations (control volume centers): $y_i = (i - \frac{1}{2}) \Delta y, \quad i = 1, \dots, 5$

Boundary conditions: $u(0, t) = U_0 = 1, \quad u(h, t) = 0$

2.3 Coefficient A

$$A = \frac{\nu \Delta t}{(\Delta y)^2} = \frac{10^{-6} \Delta t}{(0.002)^2} = 0.25 \Delta t$$

2.4 Governing Equations

Node 1 (adjacent to moving plate):

$$u_1^{n+1} = u_1^n + 0.5 \Delta t (u_2^n - 3u_1^n + 2)$$

Interior nodes ($i = 2, 3, 4$):

$$u_i^{n+1} = u_i^n + 0.25 \Delta t (u_{i+1}^n - 2u_i^n + u_{i-1}^n)$$

Node 5 (adjacent to stationary plate):

$$u_5^{n+1} = u_5^n + 0.5 \Delta t (u_4^n - 3u_5^n)$$

All equations

$$\begin{aligned} u_1^{n+1} &= u_1^n + 0.5 \Delta t (u_2^n - 3u_1^n + 2) \\ u_2^{n+1} &= u_2^n + 0.25 \Delta t (u_3^n - 2u_2^n + u_1^n) \\ u_3^{n+1} &= u_3^n + 0.25 \Delta t (u_4^n - 2u_3^n + u_2^n) \\ u_4^{n+1} &= u_4^n + 0.25 \Delta t (u_5^n - 2u_4^n + u_3^n) \\ u_5^{n+1} &= u_5^n + 0.5 \Delta t (u_4^n - 3u_5^n) \end{aligned}$$

2.5 Results

With time step = 0.2, the following chart depicts the velocity profile using explicit euler discretization mechanism. The source code for the same is in the Appendix G

2.5.1 Comparison with Analytical Solution

The numerical solution was compared with the analytical solution at 1 s, 5 s, and 10 s using percentage error at each node.

- At 1 s, large errors are observed, particularly away from the moving plate. The error is 17.094% at node 1 and increases to 95.959% at node 2 and 844.829% at node 3. This occurs because the analytical velocities are extremely small at early times, making percentage errors large even for small absolute differences.
- At 5 s, the agreement improves. The error is 3.418% at node 1 and 7.090% at node 2, but remains higher farther from the moving plate, reaching 38.403% at node 4 and 130.511% at node 5.
- At 10 s, good agreement is obtained across the domain. Errors reduce to 1.290%, 2.620%, and 4.890% at nodes 1, 2, and 3 respectively, confirming convergence toward the analytical solution.
- Overall, the numerical solution becomes more accurate with time as momentum diffuses through the domain.

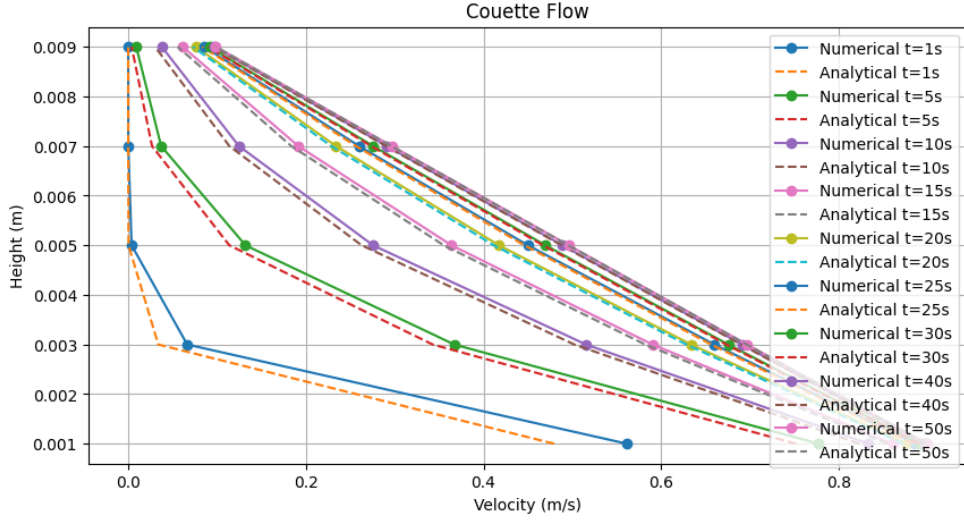
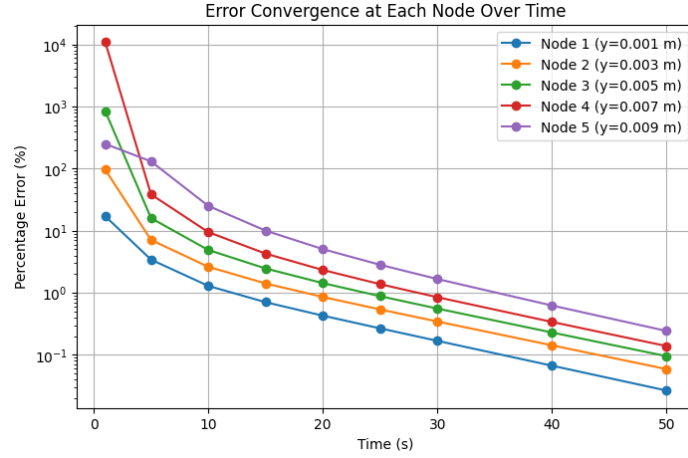


Figure 2: Explicit method - Evolution of velocity at all nodes

The percentage error values are provided in Appendix H. The improvement of error from the analytical value is presented in the graph below.



2.5.2 Stability analysis

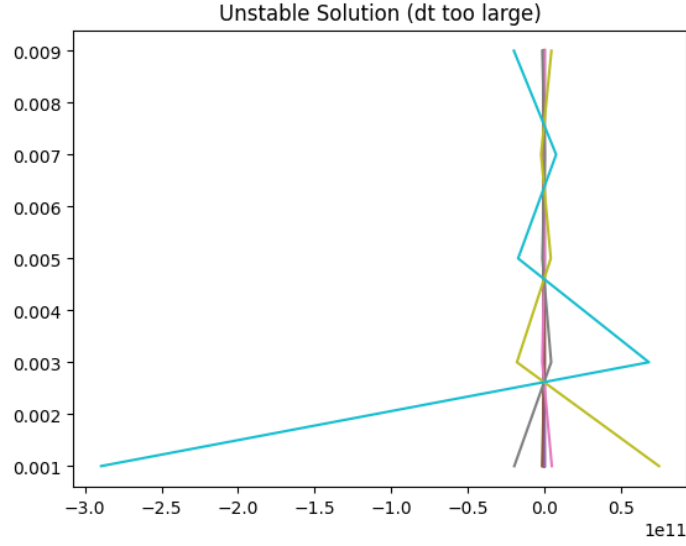
The governing equation for an interior node is as below with $A = \frac{\nu \Delta t}{(\Delta y)^2}$

$$u_i^{n+1} = (1 - 2A)u_i^n + Au_{i+1}^n + Au_{i-1}^n$$

For a stable solution, all coefficients must be non-negative. Therefore, $1 - 2A \geq 0$, which gives $A \leq \frac{1}{2}$

Substituting the definition of A , $\frac{\nu \Delta t}{(\Delta y)^2} \leq \frac{1}{2}$ Thus, the stability condition is $\Delta t \leq \frac{(\Delta y)^2}{2\nu}$

Using the given values, the numerical solution remains stable. $\Delta t \leq 2$ s. For $\Delta t = 3$, the explicit method convergence will look like below. The code for reference is at Appendix I



3 Crank–Nicolson Discretization

3.1 Governing equations

The physical parameters, grid definition, coefficient values are all same as the explicit case. The equations before substituting A are like below and more details can be found in Appendix F

Node 1 equation (adjacent to moving plate):

$$(1 + 3A)u_1^{n+1} - Au_2^{n+1} = (1 - 3A)u_1^n + Au_2^n + 4AU_0$$

Node 2 equation (interior node):

$$(1 + A)u_2^{n+1} - \frac{A}{2}u_3^{n+1} - \frac{A}{2}u_1^{n+1} = (1 - A)u_2^n + \frac{A}{2}u_3^n + \frac{A}{2}u_1^n$$

Node 3 equation (interior node):

$$(1 + A)u_3^{n+1} - \frac{A}{2}u_4^{n+1} - \frac{A}{2}u_2^{n+1} = (1 - A)u_3^n + \frac{A}{2}u_4^n + \frac{A}{2}u_2^n$$

Node 4 equation (interior node):

$$(1 + A)u_4^{n+1} - \frac{A}{2}u_5^{n+1} - \frac{A}{2}u_3^{n+1} = (1 - A)u_4^n + \frac{A}{2}u_5^n + \frac{A}{2}u_3^n$$

Node 5 equation (adjacent to stationary plate):

$$(1 + 3A)u_5^{n+1} - Au_4^{n+1} = (1 - 3A)u_5^n + Au_4^n$$

3.2 Results

In the Crank-Nicolson discretization, we need to solve the system of equations at each step. The source code for sample implementation with time step 0.2 is in Appendix J

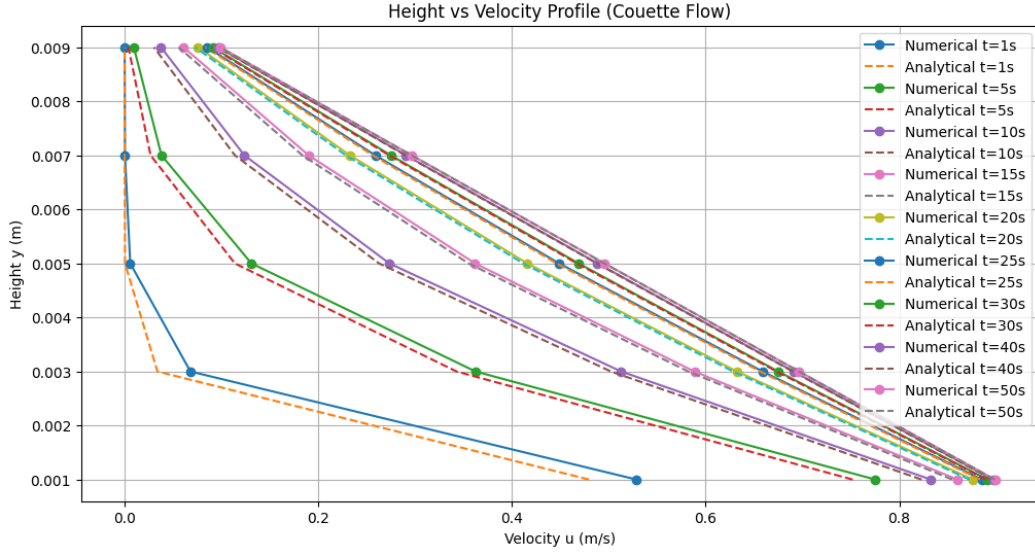
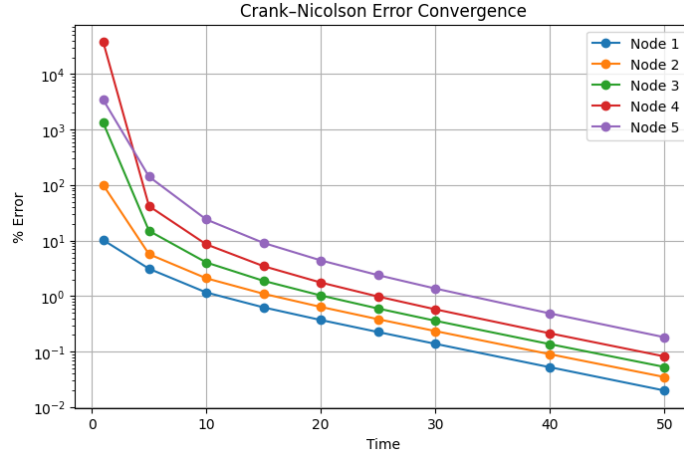


Figure 3: Velocity profile

The error table comparing the analytics is solution for each time step and node is at Appendix K. The visual representation of the error convergence with analytics solution is as in the figure below.



3.2.1 Comparison with Analytical Solution

The numerical solution obtained using the Crank–Nicolson scheme was compared with the analytical solution at 1 s, 5 s, and 10 s using percentage error at each node.

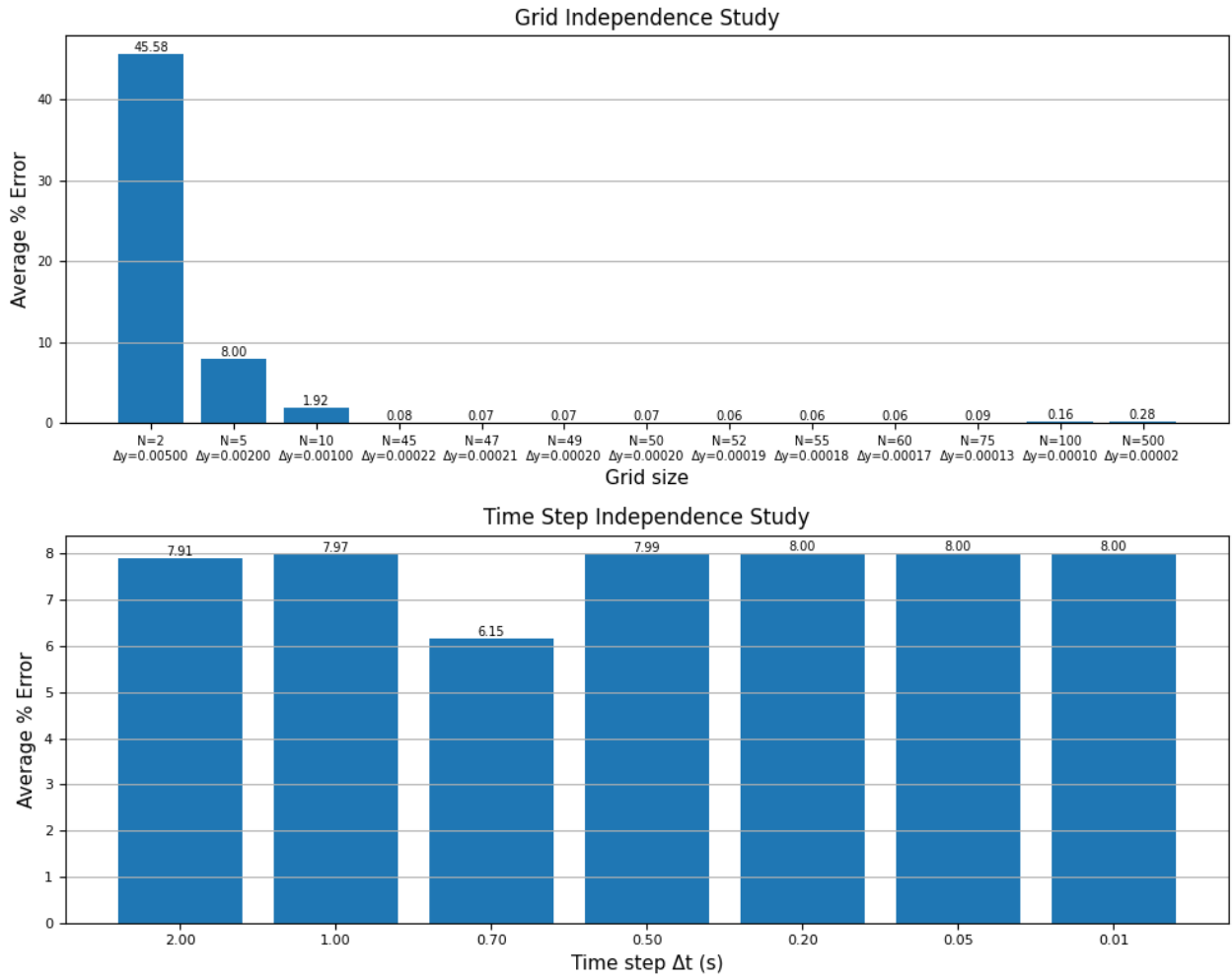
- At 1 s, large percentage errors are observed away from the moving plate. The error is 10.256% at node 1 and increases to 100.485% at node 2 and 1323.007% at node 3. Extremely large percentage errors occur at nodes 4 and 5 because the analytical velocities are nearly zero at early times, making percentage error very sensitive to small absolute differences.
- At 5 s, the agreement improves significantly. The error is 3.120% at node 1 and 5.707% at node 2, while larger errors remain farther from the moving plate, reaching 41.885% at node 4 and 142.084% at node 5.
- At 10 s, good agreement is observed across the domain. The errors reduce to 1.168%, 2.103%, and 4.050% at nodes 1, 2, and 3 respectively, indicating that the numerical solution closely follows the analytical solution.
- Overall, the Crank–Nicolson scheme shows improving accuracy with time and provides stable and accurate results as momentum diffuses through the domain.

3.2.2 Grid and time independence study

For the physical parameters and scheme implementation chosen, the following are the results of multiple grid spacing and time spacing options. As per data, $N \sim 55$ and $dt \sim 0.7$ are optimal for reducing error or deviation

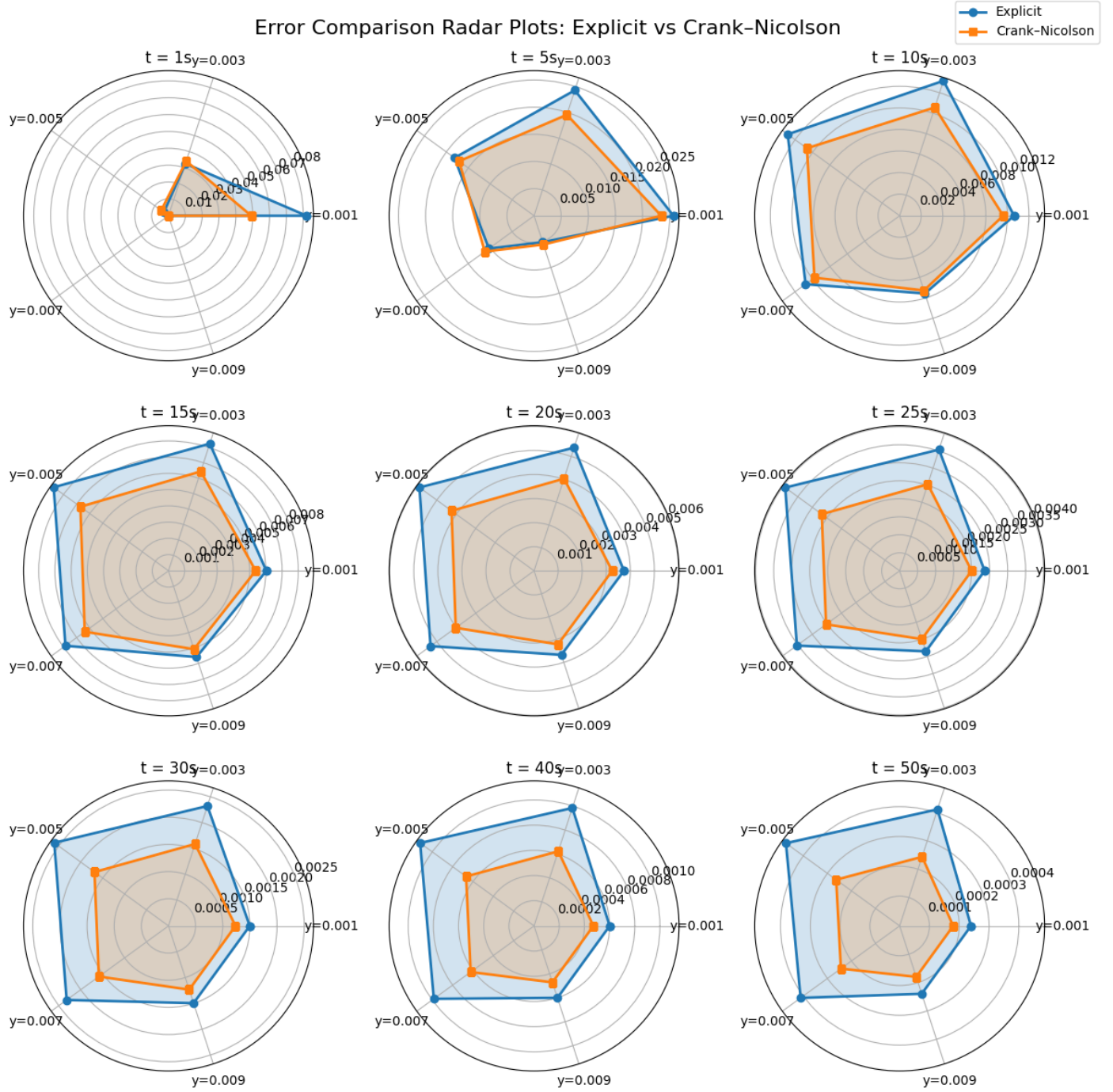
from the analytics solution.

The code that produced the results below is provided in [Appendix L](#)



A Comparison between explicit and Clark-Nicholson scheme

The following radar plots provide insight into the progression of error



B Steps for solving general equation for non-boundary nodes

The governing equation to start with is the one below.

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2}$$

Integrating over the control volume:

$$y \in \left[y_P - \frac{\Delta y}{2}, y_P + \frac{\Delta y}{2} \right]$$

and over timestep:

$$t \in [t^n, t^{n+1}]$$

gives:

$$\Delta y (u_P^{n+1} - u_P^n) = \nu \int_{t^n}^{t^{n+1}} \left[\left(\frac{\partial u}{\partial y} \right)_e - \left(\frac{\partial u}{\partial y} \right)_w \right] dt$$

The θ -method approximates the time integral as a weighted average:

$$\int_{t^n}^{t^{n+1}} f(t) dt = \Delta t [\theta f^{n+1} + (1 - \theta) f^n]$$

Applying this to the diffusion term:

$$\Delta y (u_P^{n+1} - u_P^n) = \nu \Delta t \left[\theta \left(\left(\frac{\partial u}{\partial y} \right)_e^{n+1} - \left(\frac{\partial u}{\partial y} \right)_w^{n+1} \right) + (1 - \theta) \left(\left(\frac{\partial u}{\partial y} \right)_e^n - \left(\frac{\partial u}{\partial y} \right)_w^n \right) \right]$$

Using central differences:

$$\begin{aligned} \left(\frac{\partial u}{\partial y} \right)_e &= \frac{u_E - u_P}{\Delta y} \\ \left(\frac{\partial u}{\partial y} \right)_w &= \frac{u_P - u_W}{\Delta y} \end{aligned}$$

Substituting:

$$\Delta y (u_P^{n+1} - u_P^n) = \frac{\nu \Delta t}{\Delta y} [\theta (u_E^{n+1} - 2u_P^{n+1} + u_W^{n+1}) + (1 - \theta) (u_E^n - 2u_P^n + u_W^n)]$$

Dividing by

$$\Delta y$$

:

$$u_P^{n+1} - u_P^n = A [\theta (u_E^{n+1} - 2u_P^{n+1} + u_W^{n+1}) + (1 - \theta) (u_E^n - 2u_P^n + u_W^n)]$$

where:

$$A = \frac{\nu \Delta t}{(\Delta y)^2}$$

Rearranging into standard algebraic form:

$$(1 + 2\theta A)u_P^{n+1} - \theta A u_E^{n+1} - \theta A u_W^{n+1} = (1 - 2(1 - \theta)A)u_P^n + (1 - \theta)A u_E^n + (1 - \theta)A u_W^n$$

C Steps for solving boundary with moving plate

The governing equation is:

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2}$$

The moving plate coincides with the east face, where velocity is prescribed:

$$u_e = U_0$$

The boundary control volume extends from the plate to the interior face:

$$y \in \left[y_e - \frac{\Delta y}{2}, y_e \right]$$

The control volume thickness is:

$$V = \frac{\Delta y}{2}$$

Integrating over the control volume and timestep:

$$\frac{\Delta y}{2} (u_P^{n+1} - u_P^n) = \nu \int_{t^n}^{t^{n+1}} \left[\left(\frac{\partial u}{\partial y} \right)_e - \left(\frac{\partial u}{\partial y} \right)_w \right] dt$$

Applying the θ -method:

$$= \nu \Delta t \left[\theta \left(\left(\frac{\partial u}{\partial y} \right)_e^{n+1} - \left(\frac{\partial u}{\partial y} \right)_w^{n+1} \right) + (1 - \theta) \left(\left(\frac{\partial u}{\partial y} \right)_e^n - \left(\frac{\partial u}{\partial y} \right)_w^n \right) \right]$$

The east face gradient uses the prescribed wall velocity:

$$\left(\frac{\partial u}{\partial y} \right)_e = \frac{U_0 - u_P}{\Delta y/2}$$

The west face gradient uses the interior node:

$$\left(\frac{\partial u}{\partial y} \right)_w = \frac{u_P - u_W}{\Delta y}$$

Substituting:

$$\frac{\Delta y}{2} (u_P^{n+1} - u_P^n) = \frac{\nu \Delta t}{\Delta y} [\theta (2U_0 - 3u_P^{n+1} + u_W^{n+1}) + (1 - \theta) (2U_0 - 3u_P^n + u_W^n)]$$

Dividing by $\frac{\Delta y}{2}$:

$$u_P^{n+1} - u_P^n = 2A [\theta (2U_0 - 3u_P^{n+1} + u_W^{n+1}) + (1 - \theta) (2U_0 - 3u_P^n + u_W^n)]$$

where:

$$A = \frac{\nu \Delta t}{(\Delta y)^2}$$

Rearranging into algebraic form:

$$(1 + 6\theta A)u_P^{n+1} - 2\theta A u_W^{n+1} = (1 - 6(1 - \theta)A)u_P^n + 2(1 - \theta)A u_W^n + 4AU_0$$

Explicit Euler: ($\theta = 0$)

$$u_P^{n+1} = u_P^n + 2A (2U_0 - 3u_P^n + u_W^n)$$

Implicit Euler: ($\theta = 1$)

$$(1 + 6A)u_P^{n+1} - 2A u_W^{n+1} = u_P^n + 4AU_0$$

Crank–Nicolson: ($\theta = \frac{1}{2}$)

$$(1 + 3A)u_P^{n+1} - A u_W^{n+1} = (1 - 3A)u_P^n + A u_W^n + 4AU_0$$

D Steps for solving equation at static plate

The governing equation is:

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2}$$

The static plate coincides with the west face, where velocity is prescribed: $u_w = 0$

The boundary control volume extends from the interior face to the plate:

$$y \in \left[y_w, y_w + \frac{\Delta y}{2} \right]$$

The control volume thickness is: $V = \frac{\Delta y}{2}$

Integrating over the control volume and timestep:

$$\frac{\Delta y}{2} (u_P^{n+1} - u_P^n) = \nu \int_{t^n}^{t^{n+1}} \left[\left(\frac{\partial u}{\partial y} \right)_e - \left(\frac{\partial u}{\partial y} \right)_w \right] dt$$

Applying the θ -method:

$$= \nu \Delta t \left[\theta \left(\left(\frac{\partial u}{\partial y} \right)_e^{n+1} - \left(\frac{\partial u}{\partial y} \right)_w^{n+1} \right) + (1 - \theta) \left(\left(\frac{\partial u}{\partial y} \right)_e^n - \left(\frac{\partial u}{\partial y} \right)_w^n \right) \right]$$

The east face gradient uses the interior node:

$$\left(\frac{\partial u}{\partial y} \right)_e = \frac{u_E - u_P}{\Delta y}$$

The west face gradient uses the prescribed wall velocity:

$$\left(\frac{\partial u}{\partial y} \right)_w = \frac{u_P - 0}{\Delta y/2}$$

Substituting:

$$\frac{\Delta y}{2} (u_P^{n+1} - u_P^n) = \frac{\nu \Delta t}{\Delta y} [\theta (u_E^{n+1} - 3u_P^{n+1}) + (1 - \theta) (u_E^n - 3u_P^n)]$$

Dividing by $\frac{\Delta y}{2}$:

$$u_P^{n+1} - u_P^n = 2A [\theta (u_E^{n+1} - 3u_P^{n+1}) + (1 - \theta) (u_E^n - 3u_P^n)]$$

where:

$$A = \frac{\nu \Delta t}{(\Delta y)^2}$$

Rearranging into algebraic form:

$$(1 + 6\theta A)u_P^{n+1} - 2\theta A u_E^{n+1} = (1 - 6(1 - \theta)A)u_P^n + 2(1 - \theta)A u_E^n$$

Explicit Euler: ($\theta = 0$)

$$u_P^{n+1} = u_P^n + 2A (u_E^n - 3u_P^n)$$

Implicit Euler: ($\theta = 1$)

$$(1 + 6A)u_P^{n+1} - 2A u_E^{n+1} = u_P^n$$

Crank–Nicolson: ($\theta = \frac{1}{2}$)

$$(1 + 3A)u_P^{n+1} - A u_E^{n+1} = (1 - 3A)u_P^n + A u_E^n$$

E Theta Method Overview

The θ -method is a general time discretization scheme used to solve transient differential equations. It provides a unified framework that includes both explicit and implicit schemes as special cases. The method is applied to equations of the form

$$\frac{\partial \phi}{\partial t} = F(\phi, t)$$

where ϕ is the dependent variable and F represents spatial operators such as diffusion or convection. In the θ -method, the time derivative is approximated over a time step Δt as

$$\frac{\phi^{n+1} - \phi^n}{\Delta t} = \theta F(\phi^{n+1}, t^{n+1}) + (1 - \theta) F(\phi^n, t^n)$$

where

- ϕ^n is the value at the current time level,
- ϕ^{n+1} is the value at the next time level,
- θ is a weighting parameter, with $0 \leq \theta \leq 1$.

The choice of θ determines the nature of the numerical scheme:

Explicit Scheme ($\theta = 0$) When $\theta = 0$, the method becomes fully explicit:

$$\frac{\phi^{n+1} - \phi^n}{\Delta t} = F(\phi^n, t^n)$$

Implicit Scheme ($\theta = 1$) When $\theta = 1$, the method becomes fully implicit:

$$\frac{\phi^{n+1} - \phi^n}{\Delta t} = F(\phi^{n+1}, t^{n+1})$$

Crank–Nicolson Scheme ($\theta = 0.5$) When $\theta = 0.5$, the method becomes the Crank–Nicolson scheme:

$$\frac{\phi^{n+1} - \phi^n}{\Delta t} = \frac{1}{2} [F(\phi^{n+1}, t^{n+1}) + F(\phi^n, t^n)]$$

The θ -method provides flexibility of control over stability and accuracy by adjusting the parameter θ , making it particularly useful for transient diffusion problems such as unsteady Couette flow.

θ	Scheme Type	Properties
0	Explicit (Forward Euler)	Simple, conditionally stable
0.5	Crank–Nicolson	Second-order accurate, stable
1	Implicit (Backward Euler)	Unconditionally stable

F Crank–Nicolson Scheme Derivation

Crank–Nicolson corresponds to $\theta = \frac{1}{2}$

Node 1 equation (adjacent to moving plate):

$$(1 + 3A)u_1^{n+1} - Au_2^{n+1} = (1 - 3A)u_1^n + Au_2^n + 4AU_0$$

Substitute numerical value of A and $U_0 = 1$:

$$(1 + 0.75\Delta t)u_1^{n+1} - 0.25\Delta t u_2^{n+1} = (1 - 0.75\Delta t)u_1^n + 0.25\Delta t u_2^n + \Delta t$$

Node 2 equation (interior node):

$$(1 + A)u_2^{n+1} - \frac{A}{2}u_3^{n+1} - \frac{A}{2}u_1^{n+1} = (1 - A)u_2^n + \frac{A}{2}u_3^n + \frac{A}{2}u_1^n$$

Substitute numerical value of A:

$$(1 + 0.25\Delta t)u_2^{n+1} - 0.125\Delta t u_3^{n+1} - 0.125\Delta t u_1^{n+1} = (1 - 0.25\Delta t)u_2^n + 0.125\Delta t u_3^n + 0.125\Delta t u_1^n$$

Node 3 equation (interior node):

$$(1 + A)u_3^{n+1} - \frac{A}{2}u_4^{n+1} - \frac{A}{2}u_2^{n+1} = (1 - A)u_3^n + \frac{A}{2}u_4^n + \frac{A}{2}u_2^n$$

Substitute numerical value of A:

$$(1 + 0.25\Delta t)u_3^{n+1} - 0.125\Delta t u_4^{n+1} - 0.125\Delta t u_2^{n+1} = (1 - 0.25\Delta t)u_3^n + 0.125\Delta t u_4^n + 0.125\Delta t u_2^n$$

Node 4 equation (interior node):

$$(1 + A)u_4^{n+1} - \frac{A}{2}u_5^{n+1} - \frac{A}{2}u_3^{n+1} = (1 - A)u_4^n + \frac{A}{2}u_5^n + \frac{A}{2}u_3^n$$

Substitute numerical value of A:

$$(1 + 0.25\Delta t)u_4^{n+1} - 0.125\Delta t u_5^{n+1} - 0.125\Delta t u_3^{n+1} = (1 - 0.25\Delta t)u_4^n + 0.125\Delta t u_5^n + 0.125\Delta t u_3^n$$

Node 5 equation (adjacent to stationary plate):

$$(1 + 3A)u_5^{n+1} - Au_4^{n+1} = (1 - 3A)u_5^n + Au_4^n$$

Substitute numerical value of A:

$$(1 + 0.75\Delta t)u_5^{n+1} - 0.25\Delta t u_4^{n+1} = (1 - 0.75\Delta t)u_5^n + 0.25\Delta t u_4^n$$

G Explicit method python code

```
import numpy as np
import matplotlib.pyplot as plt

time_list = [1,5,10,15,20,25,30,40,50]

t_max = max(time_list)

# -----
# Physical parameters
# -----
h = 0.01
U0 = 1.0
nu = 1e-6

# -----
# Grid (5 CV centers)
# -----
dy = h/5

y = np.array([
    dy/2,
    3*dy/2,
    5*dy/2,
    7*dy/2,
    9*dy/2
])

# -----
# Time step
# -----
dt = 0.2
A = nu*dt/dy**2

print("A□=", A)

# -----
# Analytical solution
# -----
def analytical(y, t, terms=200):

    result = U0*(h-y)/h

    sum_term = np.zeros_like(y)

    for n in range(1, terms+1):

        sum_term += (
            (1/n)
            * np.sin(n*np.pi*y/h)
            * np.exp(-n**2*np.pi**2*nu*t/h**2)
        )

    result -= (2*U0/np.pi)*sum_term

    return result

# -----
# Numerical solution using YOUR equations
# -----
u_num = np.zeros(5)
```

```

profiles_num = {}
profiles_ana = {}

profiles_num[0] = u_num.copy()
profiles_ana[0] = analytical(y,0)

steps = int(t_max/dt)

for step in range(1, steps+1):
    u_old = u_num.copy()

    # bottom boundary node (moving plate)
    u_num[0] = (
        u_old[0]
        + 2*A*(u_old[1] - 3*u_old[0] + 2*U0)
    )

    # interior nodes
    u_num[1] = u_old[1] + A*(u_old[2] - 2*u_old[1] + u_old[0])
    u_num[2] = u_old[2] + A*(u_old[3] - 2*u_old[2] + u_old[1])
    u_num[3] = u_old[3] + A*(u_old[4] - 2*u_old[3] + u_old[2])

    # top boundary node (stationary plate)
    u_num[4] = (
        u_old[4]
        + 2*A*(u_old[3] - 3*u_old[4])
    )

    t = step*dt

    if any(abs(t - target) < 1e-12 for target in time_list):
        profiles_num[t] = u_num.copy()
        profiles_ana[t] = analytical(y,t)

# -----
# Plot comparison
# -----
plt.figure(figsize=(10,5))

for t in time_list:
    plt.plot(profiles_num[t],y,'o-',label=f"Numerical_t={t}s")
    plt.plot(profiles_ana[t],y,'--',label=f"Analytical_t={t}s")

plt.xlabel("Velocity (m/s)")
plt.ylabel("Height (m)")
plt.title("Couette Flow")
plt.legend()
plt.grid()
plt.show()

# -----
# Error table
# -----
print("\nPercentage Error Table\n")
print("Time (s) | Node | y (m) | Numerical | Analytical | % Error")
print("-----")

for t in time_list:
    num = profiles_num[t]
    ana = profiles_ana[t]

    for i in range(5):
        if abs(ana[i]) > 1e-12:

```

```

        error = abs((num[i]-ana[i])/max(ana[i], 1e-6))*100
    else:
        error = 0

    print(f"{t:7.1f}|_{i+1:4d}|_{y[i]:.4f}|_{f"{num[i]:.6f}|_{ana[i]:.6f}|_{error:7.3f}"")

print("-----")

```

H Percentage error - Explicit method

Time (s)	Node	y (m)	Numerical	Analytical	% Error
----------	------	-------	-----------	------------	---------

1.0	1	0.0010	0.561468	0.479500	17.094
1.0	2	0.0030	0.066420	0.033895	95.959
1.0	3	0.0050	0.003845	0.000407	844.829
1.0	4	0.0070	0.000110	0.000001	10925.690
1.0	5	0.0090	0.000003	0.000000	249.980
5.0	1	0.0010	0.777526	0.751830	3.418
5.0	2	0.0030	0.367085	0.342782	7.090
5.0	3	0.0050	0.131984	0.113844	15.934
5.0	4	0.0070	0.037116	0.026817	38.403
5.0	5	0.0090	0.009041	0.003922	130.511
10.0	1	0.0010	0.833662	0.823044	1.290
10.0	2	0.0030	0.515350	0.502191	2.620
10.0	3	0.0050	0.275604	0.262756	4.890
10.0	4	0.0070	0.124665	0.113874	9.476
10.0	5	0.0090	0.037850	0.030265	25.060
15.0	1	0.0010	0.860767	0.854736	0.706
15.0	2	0.0030	0.590233	0.581998	1.415
15.0	3	0.0050	0.363896	0.355146	2.464
15.0	4	0.0070	0.191472	0.183621	4.276
15.0	5	0.0090	0.061324	0.055739	10.021
20.0	1	0.0010	0.876339	0.872603	0.428
20.0	2	0.0030	0.633725	0.628343	0.857
20.0	3	0.0050	0.417459	0.411566	1.432
20.0	4	0.0070	0.233890	0.228568	2.328
20.0	5	0.0090	0.076413	0.072742	5.046
25.0	1	0.0010	0.885668	0.883307	0.267
25.0	2	0.0030	0.659846	0.656307	0.539
25.0	3	0.0050	0.449943	0.446011	0.881
25.0	4	0.0070	0.259868	0.256338	1.377
25.0	5	0.0090	0.085678	0.083326	2.822
30.0	1	0.0010	0.891311	0.889813	0.168
30.0	2	0.0030	0.675654	0.673333	0.345
30.0	3	0.0050	0.469642	0.467040	0.557
30.0	4	0.0070	0.275657	0.273337	0.849
30.0	5	0.0090	0.091312	0.089816	1.665
40.0	1	0.0010	0.896804	0.896204	0.067
40.0	2	0.0030	0.691046	0.690062	0.143

40.0		3		0.0050		0.488835		0.487716		0.229
40.0		4		0.0070		0.291046		0.290062		0.339
40.0		5		0.0090		0.096804		0.096204		0.624

50.0		1		0.0010		0.898825		0.898585		0.027
50.0		2		0.0030		0.696707		0.696296		0.059
50.0		3		0.0050		0.495894		0.495422		0.095
50.0		4		0.0070		0.296707		0.296296		0.139
50.0		5		0.0090		0.098825		0.098585		0.243

I Unstable dt=3, explicit method

```
# unstable test
dt_unstable = 3
A_unstable = nu*dt_unstable/dy**2

u = np.zeros(5)

history = []

for step in range(20):

    u_old = u.copy()

    u[0] = u_old[0] + 2*A_unstable*(u_old[1] - 3*u_old[0] + 2*U0)
    u[1] = u_old[1] + A_unstable*(u_old[2] - 2*u_old[1] + u_old[0])
    u[2] = u_old[2] + A_unstable*(u_old[3] - 2*u_old[2] + u_old[1])
    u[3] = u_old[3] + A_unstable*(u_old[4] - 2*u_old[3] + u_old[2])
    u[4] = u_old[4] + 2*A_unstable*(u_old[3] - 3*u_old[4])

    history.append(u.copy())

# plot
plt.figure()

for i in range(len(history)):
    plt.plot(history[i], y)

plt.title("Unstable_Solution_(dt_too_large)")
plt.show()
```

J Crank-Nicolson python code

```
import numpy as np
import matplotlib.pyplot as plt

# -----
# Physical parameters
# -----

h = 0.01
U0 = 1.0
mu = 1e-3
rho = 1000

nu = mu/rho

# -----
# Grid
```



```

# -----
N = 5
dy = h/N

y = np.array([(i+0.5)*dy for i in range(N)])

# -----
# Time parameters
# -----

dt = 0.2
steps = int(t_max/dt)

# -----
# Crank Nicolson coefficient
# -----

A = nu*dt/(dy**2)

# -----
# Initial condition
# -----

u_num = np.zeros(N)

# storage
profiles_cn = {}
profiles_ana = {}

# -----
# Build constant matrix M
# -----

M = np.zeros((N,N))

# moving plate boundary
M[0,0] = 1 + 3*A
M[0,1] = -A

# interior nodes
for i in range(1,N-1):

    M[i,i] = 1 + A
    M[i,i-1] = -A/2
    M[i,i+1] = -A/2

# static plate boundary
M[N-1,N-1] = 1 + 3*A
M[N-1,N-2] = -A

# -----
# Time loop
# -----

for step in range(1,steps+1):

    u_old = u_num.copy()

    R = np.zeros((N,N))
    b = np.zeros(N)

    # moving plate boundary
    R[0,0] = 1 - 3*A

```

```

R[0,1] = A
b[0] = 4*A*U0

# interior nodes
for i in range(1,N-1):

    R[i,i] = 1 - A
    R[i,i-1] = A/2
    R[i,i+1] = A/2

# static plate boundary
R[N-1,N-1] = 1 - 3*A
R[N-1,N-2] = A

rhs = R @ u_old + b

# solve system
u_num = np.linalg.solve(M, rhs)

t = step*dt

if t in time_list:

    profiles_cn[t] = u_num.copy()
    profiles_ana[t] = analytical(y,t)

# -----
# Height vs Velocity plot
# -----

plt.figure(figsize=(12,6))

for t in time_list:

    num = profiles_cn[t]
    ana = profiles_ana[t]

    plt.plot(num, y, 'o-', label=f'Numerical_t={t}s')
    plt.plot(ana, y, '--', label=f'Analytical_t={t}s')

plt.xlabel('Velocity_u(m/s)')
plt.ylabel('Height_y(m)')
plt.title('Height vs Velocity Profile (Couette Flow)')
plt.legend()
plt.grid()

plt.show()

# -----
# Error table
# -----
print("\nPercentage Error Table\n")
print("Time(s) | Node | y(m) | Numerical | Analytical | % Error")
print("-----")

for t in time_list:
    num = profiles_cn[t]
    ana = profiles_ana[t]

    for i in range(5):
        if abs(ana[i]) > 1e-12:
            error = abs((num[i]-ana[i])/max(ana[i], 1e-6))*100
        else:

```

```

error = 0

print(f"{t:7.1f}|_{i+1:4d}|_{y[i]:.4f}|_"
      f"{num[i]:.6f}|_{ana[i]:.6f}|_{error:7.3f}")

print("-----")

```

K Percentage Error Crank Nicolson

Time (s)	Node	y (m)	Numerical	Analytical	% Error

1.0	1	0.0010	0.528679	0.479500	10.256
1.0	2	0.0030	0.067954	0.033895	100.485
1.0	3	0.0050	0.005791	0.000407	1323.007
1.0	4	0.0070	0.000379	0.000001	37817.578
1.0	5	0.0090	0.000034	0.000000	3444.859

5.0	1	0.0010	0.775285	0.751830	3.120
5.0	2	0.0030	0.362345	0.342782	5.707
5.0	3	0.0050	0.130941	0.113844	15.017
5.0	4	0.0070	0.038050	0.026817	41.885
5.0	5	0.0090	0.009495	0.003922	142.084

10.0	1	0.0010	0.832657	0.823044	1.168
10.0	2	0.0030	0.512753	0.502191	2.103
10.0	3	0.0050	0.273398	0.262756	4.050
10.0	4	0.0070	0.123648	0.113874	8.582
10.0	5	0.0090	0.037554	0.030265	24.084

15.0	1	0.0010	0.860110	0.854736	0.629
15.0	2	0.0030	0.588435	0.581998	1.106
15.0	3	0.0050	0.361859	0.355146	1.890
15.0	4	0.0070	0.190002	0.183621	3.475
15.0	5	0.0090	0.060814	0.055739	9.106

20.0	1	0.0010	0.875853	0.872603	0.372
20.0	2	0.0030	0.632370	0.628343	0.641
20.0	3	0.0050	0.415807	0.411566	1.030
20.0	4	0.0070	0.232595	0.228568	1.762
20.0	5	0.0090	0.075954	0.072742	4.415

25.0	1	0.0010	0.885307	0.883307	0.226
25.0	2	0.0030	0.658834	0.656307	0.385
25.0	3	0.0050	0.448687	0.446011	0.600
25.0	4	0.0070	0.258867	0.256338	0.986
25.0	5	0.0090	0.085321	0.083326	2.394

30.0	1	0.0010	0.891048	0.889813	0.139
30.0	2	0.0030	0.674918	0.673333	0.236
30.0	3	0.0050	0.468727	0.467040	0.361
30.0	4	0.0070	0.274923	0.273337	0.580
30.0	5	0.0090	0.091050	0.089816	1.374

40.0	1	0.0010	0.896675	0.896204	0.053
40.0	2	0.0030	0.690684	0.690062	0.090

40.0		3		0.0050		0.488384		0.487716		0.137
40.0		4		0.0070		0.290684		0.290062		0.215
40.0		5		0.0090		0.096675		0.096204		0.490

50.0		1		0.0010		0.898765		0.898585		0.020
50.0		2		0.0030		0.696540		0.696296		0.035
50.0		3		0.0050		0.495685		0.495422		0.053
50.0		4		0.0070		0.296540		0.296296		0.082
50.0		5		0.0090		0.098765		0.098585		0.182

L Grid and time independence study code

```
def run_cn_solver(N, dt, t_target):

    dy = h/N
    y = np.array([(i+0.5)*dy for i in range(N)])

    A = nu*dt/(dy**2)

    steps = int(t_target/dt)

    u_num = np.zeros(N)

    # build matrix M
    M = np.zeros((N,N))

    M[0,0] = 1 + 3*A
    M[0,1] = -A

    for i in range(1,N-1):
        M[i,i] = 1 + A
        M[i,i-1] = -A/2
        M[i,i+1] = -A/2

    M[N-1,N-1] = 1 + 3*A
    M[N-1,N-2] = -A

    for step in range(steps):

        u_old = u_num.copy()

        R = np.zeros((N,N))
        b = np.zeros(N)

        R[0,0] = 1 - 3*A
        R[0,1] = A
        b[0] = 4*A*u0

        for i in range(1,N-1):
            R[i,i] = 1 - A
            R[i,i-1] = A/2
            R[i,i+1] = A/2

        R[N-1,N-1] = 1 - 3*A
        R[N-1,N-2] = A

        rhs = R @ u_old + b

        u_num = np.linalg.solve(M, rhs)

    ana = analytical(y, t_target)
```

```

    error = np.mean(np.abs((u_num - ana)/ana))*100

    return dy, error

N_list = [2,5,10,15,50,100,500,700 ,1000]

dy_vals = []
error_dy_vals = []

for N in N_list:

    dy, err = run_cn_solver(N, dt=0.2, t_target=10)

    dy_vals.append(dy)
    error_dy_vals.append(err)

dt_list = [2, 1, 0.7, 0.5, 0.2, 0.05, 0.01]

dt_vals = []
error_vals_dt = []

for dt in dt_list:

    dy, err = run_cn_solver(N=5, dt=dt, t_target=10)

    dt_vals.append(dt)
    error_vals_dt.append(err)

fig, axes = plt.subplots(1, 2, figsize=(14,6))

# -----
# Grid independence plot
# -----

labels_dy = [f"N={N}\ndelta_y={dy:.5f}" for N, dy in zip(N_list, dy_vals)]

bars1 = axes[0].bar(labels_dy, error_dy_vals)

axes[0].set_xlabel("Grid_size", fontsize=11)
axes[0].set_ylabel("Average % Error", fontsize=11)
axes[0].set_title("Grid Independence Study", fontsize=12)

axes[0].tick_params(axis='x', labelsiz=7)
axes[0].tick_params(axis='y', labelsiz=7)

axes[0].grid(axis='y')

# smaller value labels
for bar in bars1:
    height = bar.get_height()
    axes[0].text(bar.get_x() + bar.get_width()/2,
                  height,
                  f"{height:.2f}",
                  ha='center',
                  va='bottom',
                  fontsize=7)

# -----
# Time-step independence plot
# -----

```

```

labels_dt = [f"{dt:.2f}" for dt in dt_vals]

bars2 = axes[1].bar(labels_dt, error_vals_dt)

axes[1].set_xlabel("Time_Step_delta_t(s)", fontsize=11)
axes[1].set_ylabel("Average_%_Error", fontsize=11)
axes[1].set_title("Time_Step_Independence_Study", fontsize=12)

axes[1].tick_params(axis='x', labelsiz=8)
axes[1].tick_params(axis='y', labelsiz=8)

axes[1].grid(axis='y')

# smaller value labels
for bar in bars2:
    height = bar.get_height()
    axes[1].text(bar.get_x() + bar.get_width()/2,
                 height,
                 f"{height:.2f}",
                 ha='center',
                 va='bottom',
                 fontsize=7)

plt.tight_layout()

plt.show()

```